PROJECT REPORT

AI-Based Diabetes Prediction System

Submitted by:

Shailesh Rajendra Patil

Table of Contents

Abstract 1
1. Introduction 1
2. Problem Statement 1
3. Objectives 1
4. Literature Review 2
5. Dataset Description 2
6. Exploratory Data Analysis 3
7. Methodology 4
8. Random Forest Model 5
9. Implementation 5
10. Results and Evaluation 7
11. Conclusion 8
12. Limitations and Future Scope 9
13. References 9

Abstract

This project builds a system that predicts whether a person is diabetic (1) or not (0) from everyday health information. The model is a Random Forest, which is a large group of decision trees that each vote on the answer. It was trained on a Kaggle dataset of 100,000 patient records. Every record carries 28 usable inputs: age, lifestyle habits, medical history, body measurements, cholesterol and blood sugar readings. Eighty percent of the records were used for training, and the remaining 20,000 were kept back for testing.

On those 20,000 unseen records the model reached an accuracy of 91.96% and a ROC-AUC of 0.94. It was very careful about labelling someone as diabetic: only 6 healthy people were flagged by mistake. The cost of that caution was 1,602 diabetic people who were missed, which puts recall at 86.65%. The feature importance chart showed that HbA1c and the two glucose readings account for roughly 70% of what the model relies on, while BMI, age and cholesterol count for much less. The trained model sits behind a Streamlit web app, where a user fills in a five-tab form and receives a predicted label with a probability. The dataset is synthetic, so the results should be read as a learning exercise and not as clinical evidence.

1. Introduction

Diabetes is a long-term condition in which the body cannot keep blood sugar at a healthy level. It often develops slowly, and plenty of people only learn about it when a complication has already started [4]. Spotting the risk early gives a person time to change their diet, move more and see a doctor before things get worse.

A doctor confirms diabetes with blood tests, but much of the other useful information is cheap and easy to collect: age, weight, blood pressure, family history, how active someone is. A machine learning model can look at all of these together and pick up patterns that are hard to see by eye. That is the idea behind this project. A classifier is trained on past patient records and placed inside a simple web page, so that anyone can type in their details and see a prediction within seconds.

2. Problem Statement

Given the health record of one person, the task is to decide whether that person is diabetic (class 1) or not (class 0). The decision has to come from a model that learned from earlier records and was then checked on records it had never seen. The system must also be usable by someone who has never written a line of code, so it needs a screen where the details can be entered and the answer shown together with a probability.

3. Objectives

- Train a classifier that separates diabetic from non-diabetic patients.
- Prepare the data: check for gaps, turn text columns into numbers and scale the numeric ones.
- Study the data with charts to see which factors move together with diabetes.

- Test the model on unseen records using accuracy, precision, recall, F1-score, the confusion matrix and the ROC curve.
- Find out which features the model depends on most.
- Build a Streamlit app that takes patient details and returns a label and a probability.

4. Literature Review

Early diabetes prediction work leaned on logistic regression and simple scoring tables built from a handful of risk factors such as age, body weight and family history. These are easy to explain, but they assume every factor pushes the risk up or down in a straight line, and real health data is rarely that tidy.

Tree-based methods cope with this better. A single decision tree asks a chain of questions, for example whether fasting glucose is above a certain value, and splits the patients accordingly. One tree is easily thrown off by small changes in the data, though. Breiman's Random Forest [1] gets around this by growing many trees on random samples of the rows and columns and letting them vote. The result is usually steadier than any one tree, and the forest can also rank which inputs were the most useful. In a medical setting that second point matters, because people want to know why a model gave a particular answer.

Public datasets such as the Pima Indians data and the CDC's BRFSS health survey have been used in many student and research projects on this topic. This project uses a different and larger Kaggle dataset [3] that includes laboratory values such as HbA1c and glucose. That makes it possible to see how much the blood tests add compared with lifestyle information alone.

5. Dataset Description

The data comes from the Diabetes Health Indicators Dataset on Kaggle [3]. We chose it because it was the only one we found that contains every input the project brief asks for: age, BMI, physical activity, blood pressure, cholesterol and glucose. According to its Kaggle page the data was generated with statistical methods inspired by real medical research, so these are not records of real patients.

The 28 input columns fall into five groups. **Personal:** age, gender, ethnicity, education level, income level, employment status, smoking status. **Lifestyle:** alcohol consumption, physical activity, diet score, sleep hours, screen time. **Medical history:** family history of diabetes, hypertension, cardiovascular disease. **Body measurements:** BMI, waist-to-hip ratio, systolic and diastolic blood pressure, heart rate. **Blood tests:** total, HDL and LDL cholesterol, triglycerides, fasting glucose, postprandial glucose, insulin level and HbA1c.

Two of the 31 columns were deliberately left out. The column diabetes_stage is simply the diagnosis written out in words, and diabetes_risk_score is a score calculated from the other columns. Keeping them would have let the model read the answer instead of predicting it. The target is diagnosed_diabetes.

Property	Value
Dataset	Diabetes Health Indicators Dataset (Kaggle)
Total records	100,000
Total columns	31 (28 used as inputs)
Target column	diagnosed_diabetes (0 = non-diabetic, 1 = diabetic)
Diabetic records	about 60,000 (60%)
Non-diabetic records	about 40,000 (40%)
Missing values	None
Type of data	Synthetic
Training / test split	80,000 / 20,000 records

Table 5.1: Summary of the dataset.

The two classes are not perfectly balanced, but the gap is not extreme either. About six in ten records belong to diabetic patients (Figure 5.1). The 60:40 ratio was kept in both the training and the test part of the split.

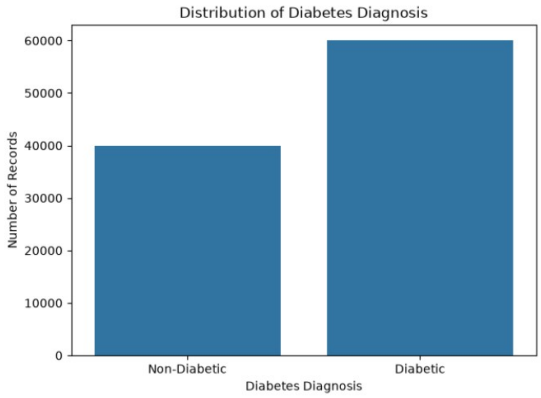


Figure 5.1: Number of diabetic and non-diabetic records in the dataset.

Table 5.2 gives the basic statistics of the main numeric columns. Two things stand out. The average HbA1c is 6.52%, which sits right on the usual diagnostic boundary of 6.5%, and the average fasting glucose is 111 mg/dL, so this population already leans towards high blood sugar. BMI also never goes above 39.2, which means very high BMI values are simply not covered by the data.

Feature	Mean	Std. dev.	Min	Median	Max
Age (years)	50.12	15.60	18	50	90
BMI (kg/m²)	25.61	3.59	15.0	25.6	39.2
Systolic BP (mmHg)	115.80	14.28	90	116	179
Diastolic BP (mmHg)	75.23	8.20	50	75	110
Fasting glucose (mg/dL)	111.12	13.60	60	111	172
Postprandial glucose (mg/dL)	160.04	30.94	70	160	287
HbA1c (%)	6.52	0.81	4.00	6.52	9.80

Table 5.2: Descriptive statistics of the main numeric columns (100,000 records).

6. Exploratory Data Analysis

Before training anything, we looked at how the main measurements differ between diabetic and non-diabetic patients. Fasting glucose shows the clearest split (Figure 6.1). The median is about 103 mg/dL for non-diabetic patients and about 117 mg/dL for diabetic ones, and the middle half of each group barely overlaps. The whiskers and outliers do overlap, so glucose on its own would not separate everybody.

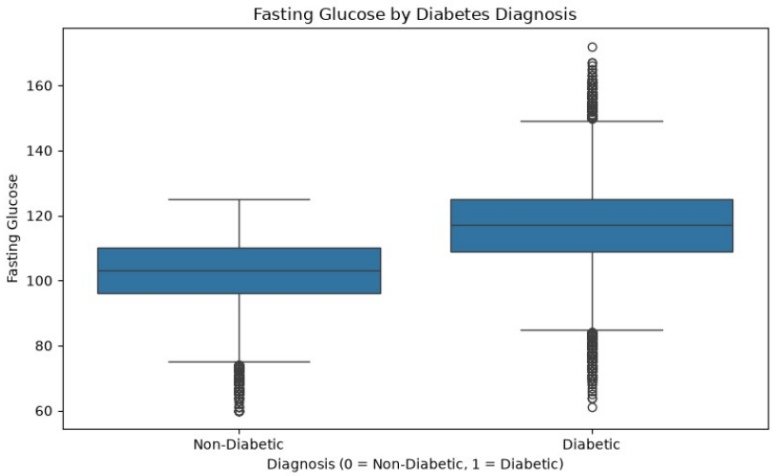


Figure 6.1: Fasting glucose for non-diabetic and diabetic patients.

BMI tells a much weaker story (Figure 6.2). The medians are about 25 and 26, and the two boxes sit almost on top of each other. In this dataset, body weight by itself says little about whether someone is diabetic.

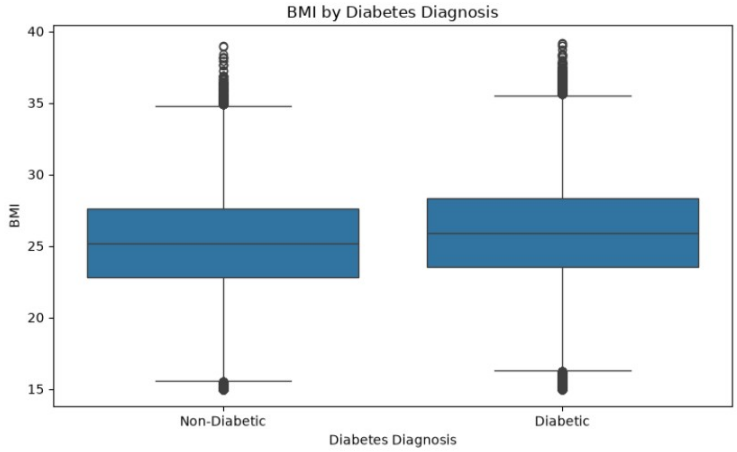


Figure 6.2: BMI for non-diabetic and diabetic patients.

The correlation heatmap (Figure 6.3) confirms both observations. Against the diagnosis, HbA1c has a correlation of 0.68, postprandial glucose 0.63 and fasting glucose 0.51. Age (0.14), BMI (0.10), systolic blood pressure (0.10), total cholesterol (0.06) and diastolic blood pressure (0.04) are all close to zero. The heatmap also shows that HbA1c and postprandial glucose are almost the same signal (0.93 with each other), and that age and systolic blood pressure move together (0.55).

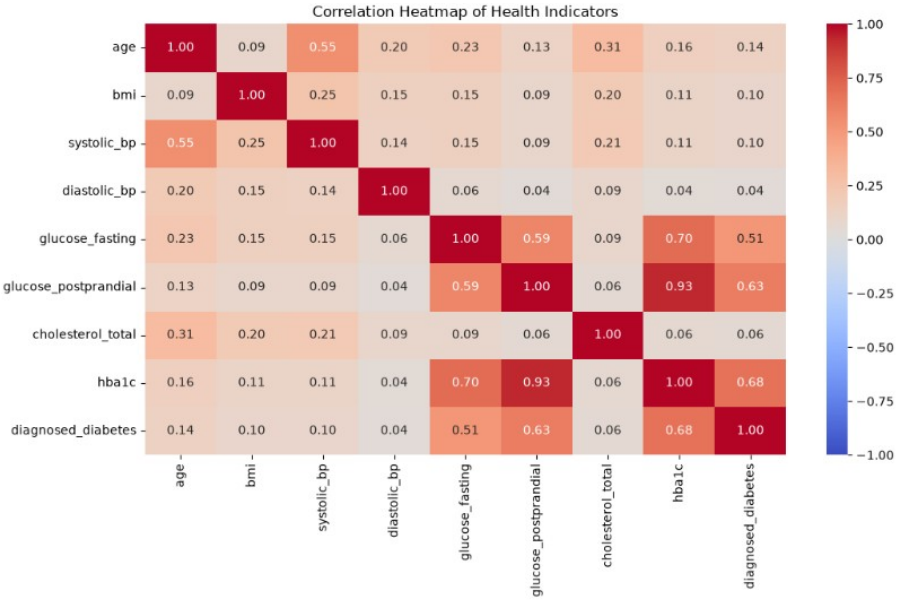


Figure 6.3: Correlation heatmap of the main numeric columns and the diagnosis.

7. Methodology

1. Load the CSV file with pandas and check its size, column types and missing values.
2. Remove diabetes_stage and diabetes_risk_score from the inputs, leaving 28 features and the target diagnosed_diabetes.
3. Explore the data with charts: class balance, box plots and a correlation heatmap.
4. Split the records 80:20 into 80,000 for training and 20,000 for testing, keeping the 60:40 class ratio in both parts.
5. Scale the numeric columns and convert the six text columns into numbers (one-hot encoding), using a preprocessor fitted on the training data.
6. Train a Random Forest classifier on the processed training data.
7. Test the model on the 20,000 unseen records and measure accuracy, precision, recall, F1-score and ROC-AUC.
8. Check which features the model relies on most.
9. Save the model and the preprocessor as .pkl files, so the Streamlit app can load them without retraining.

8. Random Forest Model

A Random Forest builds a large number of decision trees. Each tree is trained on a random sample of the training records and, at every split, is only allowed to look at a random subset of the features. Because the trees see slightly different data, they make different mistakes. When a new patient arrives, every tree gives its answer and the forest goes with the majority, and the share of trees voting for diabetes can be read as a probability.

We picked this algorithm for four reasons. It copes well with a mix of numeric and encoded text columns, it does not need the features to be perfectly scaled, it overfits less than a single deep tree, and it reports how important each feature was. The last point is what lets Section 10 say something sensible about why the model behaves the way it does.

Setting	Value
Algorithm	Random Forest classifier (scikit-learn)
Task	Binary classification (0 = non-diabetic, 1 = diabetic)
Input features	28 (22 numeric columns and 6 text columns converted to numbers)
Training records	80,000
Test records	20,000 (never seen during training)
Outputs	Predicted label and probability of class 1
Decision rule	Default setting (majority vote of the trees, about 0.5 probability)
Saved files	diabetes_model.pkl and diabetes_preprocessor.pkl

Table 8.1: Model summary.

9. Implementation

The project is written in Python. pandas and NumPy handle the data, matplotlib and seaborn draw the charts, scikit-learn provides the preprocessing, the Random Forest and the evaluation metrics, joblib saves the trained objects, and Streamlit provides the web interface [2], [5], [6]. A Jupyter notebook keeps the whole analysis, training and evaluation in one place.

The app asks for all 28 inputs, spread over five tabs: Personal, Lifestyle, Medical History, Body, and Cholesterol and Glucose. The Predict button was placed on the last tab on purpose. That way a user has to go through every section first and cannot submit a half-filled form by accident. Each field is limited to the range found in the dataset, for example BMI between 15 and 39.2, and the form is pre-filled with typical values so it is never empty.

When the button is pressed, the app puts the answers into a one-row table with exactly the same column names that were used in training, passes it through the saved preprocessor, and then asks the model for the label and the probability of class 1. The result page shows the predicted label, a gauge with the probability, a band (Low below 30%, Moderate from 30% to 60%, High above 60%), a row of summary tiles, and an expandable panel that lists every value that was submitted. A warning on the page states that the tool is for education only.

Project files:

- **app.py** – the Streamlit application.
- **diabetes_prediction.ipynb** – the notebook with analysis, training and evaluation.
- **models/diabetes_model.pkl** – the trained Random Forest.
- **models/diabetes_preprocessor.pkl** – the saved scaling and encoding steps.
- **data/** – the dataset CSV file.
- **evaluation_results.csv** – the recorded evaluation results.
- **requirements.txt** and **README.md** – packages and setup instructions.

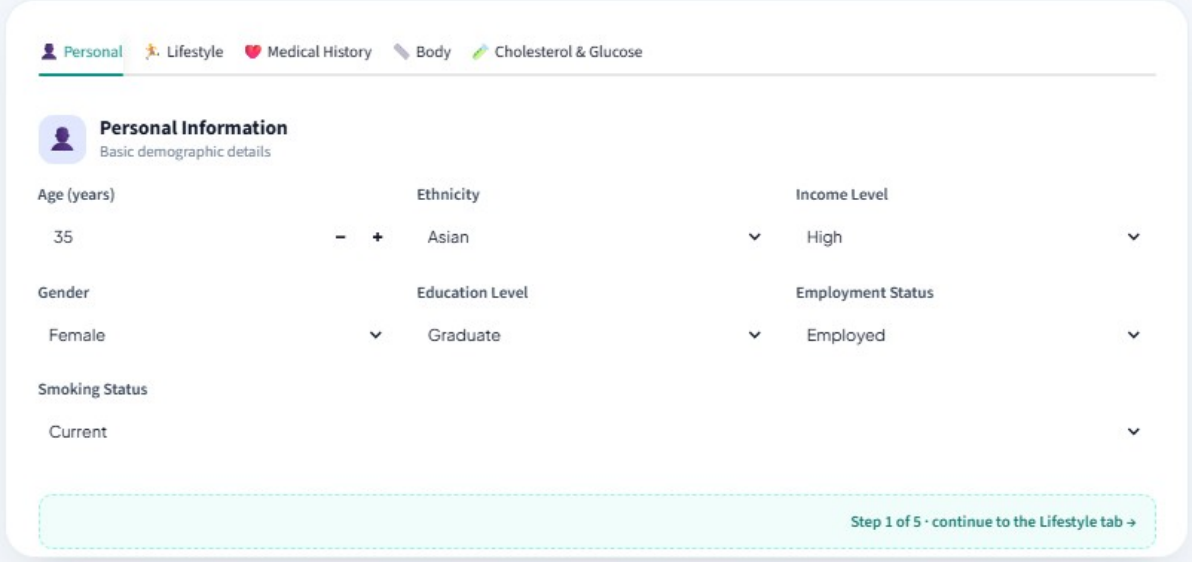


Figure 9.1: The input form, first tab (Personal). Typical values are pre-filled.

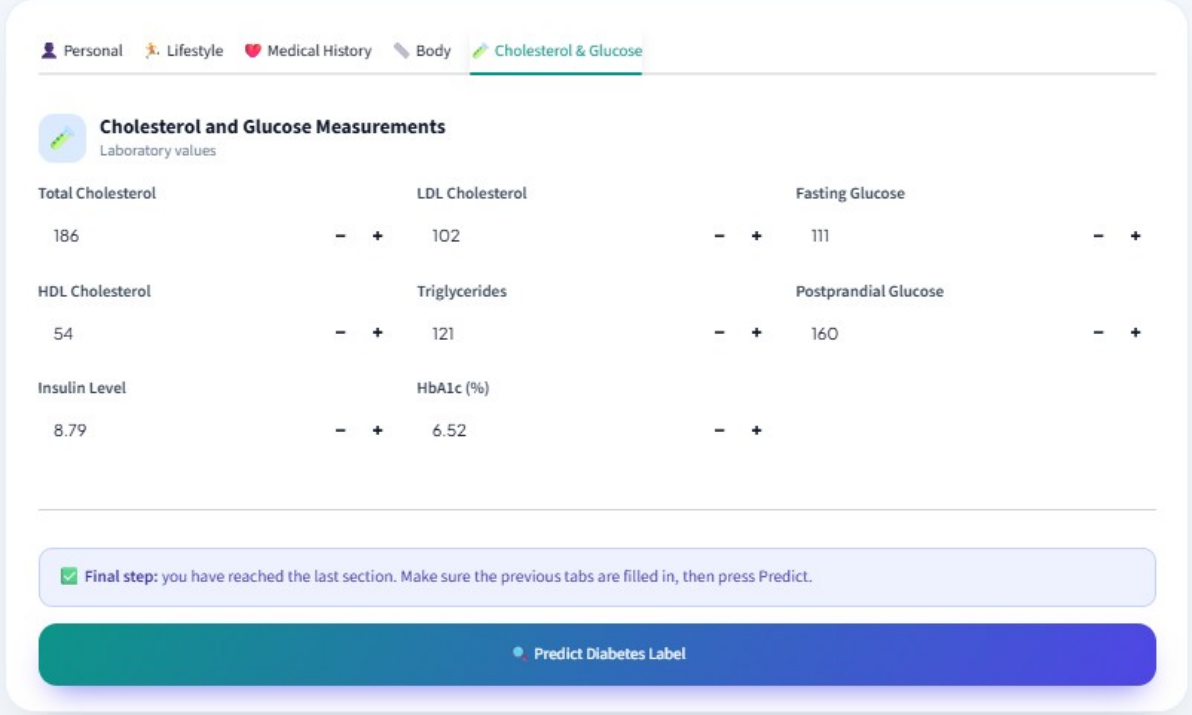


Figure 9.2: The last tab (Cholesterol and Glucose), where the Predict button appears.

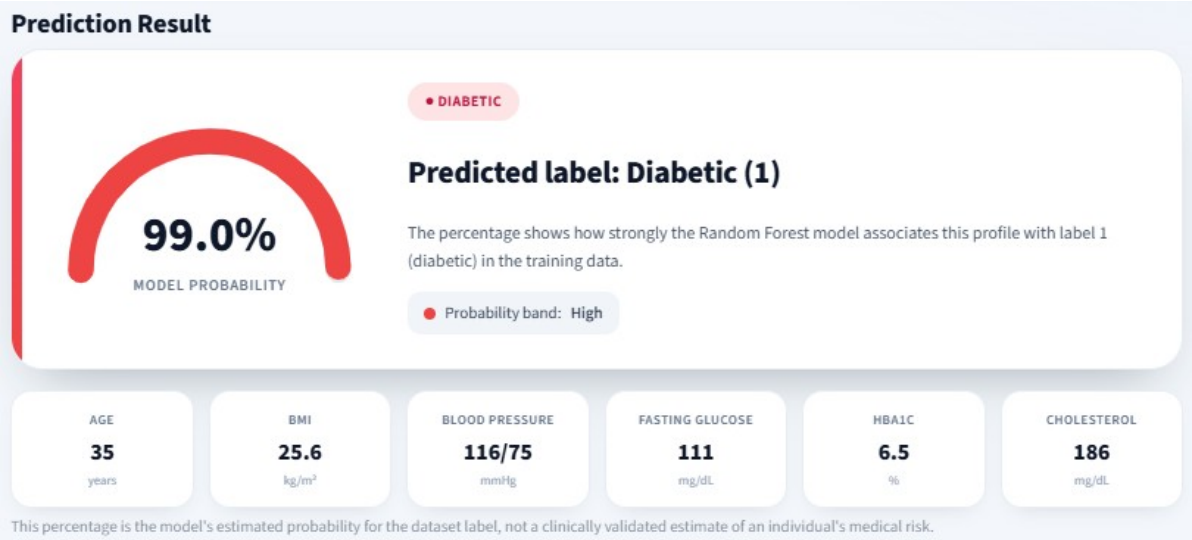


Figure 9.3: Prediction for the pre-filled values. The model labels this profile as diabetic with 99.0% probability.

The pre-filled values are typical for this dataset, and as Table 5.2 showed, typical HbA1c and glucose values here are already high. That is why the model calls this profile diabetic with such a high probability.

10. Results and Evaluation

All figures below were measured on the 20,000 test records that the model never saw during training. Figure 10.1 shows the confusion matrix and the ROC curve.

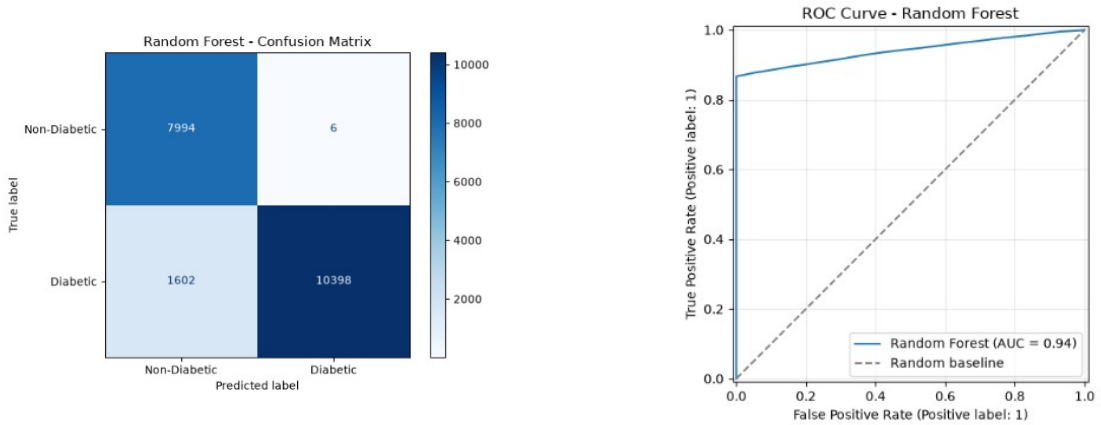


Figure 10.1: Confusion matrix (left) and ROC curve (right) of the Random Forest on the test set.

Metric	Result
Accuracy	**91.96%**
Precision (diabetic class)	99.94%
Recall (diabetic class)	86.65%
F1-score (diabetic class)	92.82%
ROC-AUC	0.94
Correct predictions	18,392 of 20,000
Incorrect predictions	1,608 of 20,000
False positives (healthy flagged as diabetic)	6
False negatives (diabetic missed)	1,602

Table 10.1: Test results (derived from the confusion matrix in Figure 10.1).

Class	Precision	Recall	F1-score	Records
Non-diabetic (0)	83.31%	99.93%	90.86%	8,000
Diabetic (1)	99.94%	86.65%	92.82%	12,000

Table 10.2: Results for each class.

What the numbers say

Of the 8,000 healthy people in the test set, 7,994 were recognised correctly and only 6 were wrongly called diabetic. When the model says a person is diabetic it is almost always right, which is where the precision of 99.94% comes from. The weak spot is on the other side. Of the 12,000 diabetic people, 1,602 were labelled healthy, so about one in eight diabetic patients (13.35%) slipped through. For a screening tool a missed case is usually worse than an unnecessary check-up, so this is the number to improve first.

The ROC curve tells the same story. It climbs almost straight up to a true positive rate of roughly 0.87 before it bends and flattens, ending at an AUC of 0.94. In plain words, the model is completely sure about most of the diabetic patients, but the rest look too much like healthy people for it to separate them with these inputs.

Which features mattered

Table 10.3 and Figure 10.2 show the importance the Random Forest assigned to each feature. The three blood sugar measurements dominate. HbA1c alone carries 0.42, postprandial glucose 0.19 and fasting glucose about 0.10, which adds up to roughly 0.71 of the total. Every other feature in the top 15 sits between 0.014 and 0.018. Physical activity, age and BMI come first among them, but the gaps are so small that their order should not be taken too seriously.

Rank	Feature	Importance	Type
1	HbA1c	0.4212	Blood test
2	Postprandial glucose	0.1896	Blood test
3	Fasting glucose	about 0.10	Blood test
4	Physical activity (min/week)	0.0177	Lifestyle
5	Age	0.0170	Personal
6	BMI	0.0161	Body measurement
7	Triglycerides	0.0156	Blood test
8	Insulin level	0.0151	Blood test
9	Total cholesterol	0.0148	Blood test
10	Screen time (hours/day)	0.0147	Lifestyle

Table 10.3: The ten most important features.

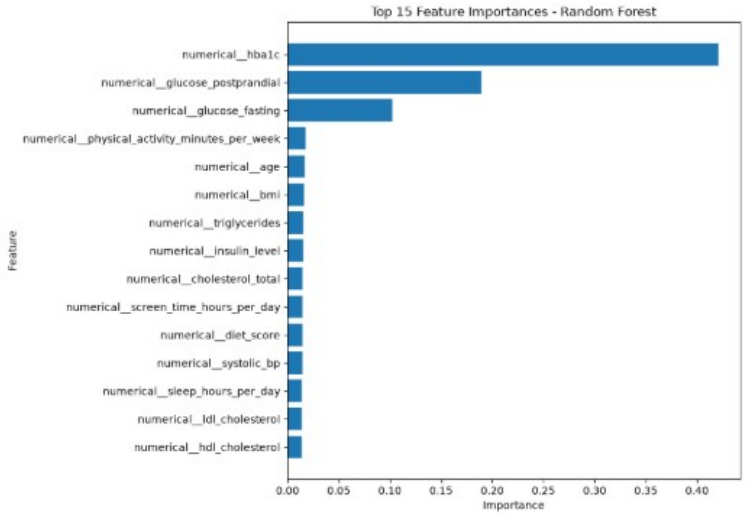


Figure 10.2: Top 15 feature importances of the Random Forest.

Two cautions go with this result. First, HbA1c and postprandial glucose are correlated at 0.93, so the forest can split the credit between them in an arbitrary way, and the exact numbers for these two should not be over-read. Second, these three tests are the very ones doctors use to confirm diabetes. A model that leans on them is partly repeating the diagnosis, and it would be much less useful for flagging someone who has not had a blood test yet.

11. Conclusion

The project delivers a working diabetes classifier and a web app around it. On 20,000 unseen records the Random Forest reached 91.96% accuracy, 99.94% precision, 86.65% recall, an F1-score of 92.82% and a ROC-AUC of 0.94. A user can fill in a five-tab form and immediately see the predicted label, a probability and a summary of the inputs.

The analysis also taught us something about the data itself. Blood sugar measurements explain most of the prediction, and the everyday factors such as BMI, age and cholesterol contribute only a little. The model is very reliable when it flags someone as diabetic, but it misses about 13% of the diabetic patients, so it is better seen as a demonstration of the method than as a screening tool that is ready to use.

12. Limitations and Future Scope

The dataset is synthetic, so the patterns the model learned may not hold for real patients. The model depends heavily on HbA1c and glucose values, which limits its use for early screening. About 13% of diabetic cases are missed. The input ranges stop at BMI 39.2 and ages 18 to 90, so people outside those ranges are not covered. The report covers a single algorithm, the Random Forest, and no other models were compared. Nothing here has been validated clinically.

Future improvements can include:

- lowering the decision threshold or using class weights to raise recall and miss fewer diabetic patients;
- training a second model without HbA1c and the glucose columns, to see how well lifestyle and body measurements alone can predict diabetes;
- comparing the Random Forest with Logistic Regression and gradient boosting models, and tuning its settings with cross-validation;
- testing the approach on real survey or clinical data such as BRFSS;
- showing a short explanation of each prediction in the app, for example with SHAP values.

13. References

[1] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[2] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[3] Thalla, M. K. (Kaggle user mohankrishnathalla). Diabetes Health Indicators Dataset. https://www.kaggle.com/datasets/mohankrishnathalla/diabetes-health-indicators-dataset

[4] World Health Organization. Diabetes (fact sheet). https://www.who.int/news-room/fact-sheets/detail/diabetes

[5] Streamlit documentation. https://docs.streamlit.io/

[6] Waskom, M. L. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

[7] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, 51–56.